lshash User Guide

This guide is a tutorial-style walkthrough for day-to-day usage.

If you want internals instead of usage, see ARCHITECTURE.md.

1. Pick an implementation

You can use either implementation:

- Bash: `./lshash.sh`
- .NET single-file binary: `dotnet/dist/linux-x64/lshash`

Both support the same core flags and are validated for behavior parity by `tests/regression.sh`.

2. Build and sanity check

Bash

```
chmod +x ./lshash.sh
./lshash.sh --help
```

.NET

```
cd dotnet
./build.sh linux-x64
../dotnet/dist/linux-x64/lshash --help
```

3. Core mental model

Think in two phases:

1. Audit phase (safe, no moves): run without `-d`
2. Remediation phase (moves duplicates): run with `-d`

This split is useful for approvals and repeatable operations.

4. Start with an audit pass

Run in your target root:

```
./lshash.sh /path/to/corpus
```

Or recursively:

```
./lshash.sh -r /path/to/corpus
```

What to look for:

- Green hashes indicate adjacent duplicate hashes in listing order.
- Summary line reports duplicate count and percent.
- Recursive mode also reports traversed directory count.

5. Narrow scope with excludes

```
./lshash.sh -r \
  -e '.git/*' \
  -e 'node_modules/*' \
  --exclude '*.tmp' \
  --exclude '*.log' \
  /path/to/corpus
```

Use this to remove noise from your audit signal before dedupe.

Note:

- Built-in exclusions are always active (for example `.lshash-exclude`, `.git/.hg/.svn`, `.gitignore`, `.mdexplore-*.json`, `*.lshash.json`, `*.tmp`, and common editor/temp files).

6. Understand dedupe scopes before moving files

The same `-d` switch behaves differently depending on scope flags:

- `-d`: per-directory contiguous duplicate runs
- `-d --directory`: full-directory hash grouping
- `-d --global`: same as `--directory` for non-recursive use
- `-d -r --global`: whole-tree hash grouping across directories

Quick scope table

Command	Scope	Sidecar JSON
`-d`	Per directory, contiguous runs	No
`-d --directory`	Per directory, full hash grouping	No
`-d --global`	Selected directory, full hash grouping	No
`-d -r --global`	Full recursive tree	Yes (`<moved-file>.lshash.json`)

7. Choose the keep policy (`MODE`)

Valid modes:

- `newer`
- `older`
- `shorter` (default)
- `longer`

Important:

- `shorter` and `longer` compare full root-relative path length (`directory path + basename`), not basename length alone.

Examples:

```
./lshash.sh -d newer /path/to/corpus
./lshash.sh -d shorter --directory /path/to/corpus
./lshash.sh -r -d longer --global /path/to/corpus
```

8. First remediation run

Recommended first cull:

```
./lshash.sh -r -d shorter /path/to/corpus
```

What happens:

- Kept file remains in place per dedupe mode.
- Other duplicates move to hidden `.dups/` folders.
- Summary changes wording to “were found and moved”.
- `.dups` directories are listed at the end.

9. Whole-tree dedupe workflow (`--global`)

Use when duplicates may exist in different directories.

```
./lshash.sh -r -d shorter --global /path/to/corpus
```

Behavior:

- Duplicate sets are formed across the entire recursive tree.
- Moved files still go to each file's source directory `.dups/`.
- A sidecar JSON is written per moved file:
  - `<moved-file>.lshash.json`
  - includes peer paths and `kept` / `moved` statuses
- In dedupe mode, directories containing `.lshash-exclude` are skipped with descendants.

Inspect sidecars:

```
find /path/to/corpus -path '*/.dups/*.lshash.json' -print
```

10. Quiet mode (`-q`)

`-q` reduces file-line output but still prints summary:

```
./lshash.sh -q /path/to/corpus
./lshash.sh -rq /path/to/corpus
./lshash.sh -r -d shorter -q /path/to/corpus
```

11. Prompt-delete mode

Post-dedupe prompt

```
./lshash.sh -d shorter --prompt-delete /path/to/corpus
```

Garbage-collect existing `.dups` trees (standalone mode)

```
./lshash.sh --prompt-delete
./lshash.sh --prompt-delete /path/to/corpus
```

Standalone `--prompt-delete` scans for existing `.dups` directories, lists them, and prompts once.

12. Archive existing `.dups` content with `--move-dups`

Use this when you want to produce a restore-ready archive from existing `.dups` content without re-running dedupe.

```
./lshash.sh --move-dups /path/to/archive
./lshash.sh --move-dups=/path/to/archive /path/to/corpus
```

Behavior:

- Recursively finds existing `.dups` directories under the selected root.
- Moves files out of `.dups` under destination using original relative paths.
- Copying the destination tree back over the source tree restores duplicates (plus sidecars, when present).
- This is a standalone mode (cannot be combined with normal scan/dedupe switches).

13. .NET runtime tuning (network-heavy scans)

When using the .NET binary, these variables are useful:

- `LSHASH_DIAGNOSTICS=1` : enable tuning diagnostics lines
- `LSHASH_HASH_WORKERS=<N>` : fixed worker count (disables adaptive worker tuning)
- `LSHASH_READ_BUFFER_KB=<N>` : read buffer size per file stream
- `LSHASH_BLAKE3_BACKEND=cpu|gpu` : backend preference (default is `cpu`)
- `LSHASH_BLAKE3_GPU_MAX_CHUNKS=<N>` : GPU chunk budget when GPU backend is used

Example:

```
LSHASH_DIAGNOSTICS=1 \
LSHASH_READ_BUFFER_KB=2048 \
dotnet/dist/linux-x64/lshash -r -d shorter --global /mnt/raid/Archives
```

Note:

- On network filesystems (for example CIFS/SMB/NFS), diagnostics are auto-printed even without `LSHASH_DIAGNOSTICS`.

14. Safe operating playbook

1. Run audit first (`-r` without `-d`).
2. Capture summary metrics.
3. Run dedupe in controlled scope.
4. Re-run audit to confirm duplicate-rate reduction.
5. Archive or clean `.dups` via `--move-dups` / `--prompt-delete` .

15. Quick command cookbook

Audit current directory

```
./lshash.sh
```

Audit recursively with SHA-256

```
./lshash.sh -r --algorithm sha256 /path/to/corpus
```

Per-directory full-group dedupe

```
./lshash.sh -d shorter --directory /path/to/corpus
```

Whole-tree dedupe and metadata sidecars

```
./lshash.sh -r -d shorter --global /path/to/corpus
```

Keep newest duplicates globally

```
./lshash.sh -r -d newer --global /path/to/corpus
```

Quiet duplicate-only output

```
./lshash.sh -rq /path/to/corpus
```

Archive `.dups`

```
./lshash.sh --move-dups /path/to/archive /path/to/corpus
```

16. Troubleshooting

Why do I see `<hash unavailable>`?

The file could not be read or hashed. The tool warns and continues.

Why did `--directory` or `--global` do nothing?

Those switches only affect behavior when dedupe is enabled (`-d`).

Why no duplicate lines in `-q` mode?

`-q` only prints duplicate lines. If no duplicates are detected, file output can be empty (summary still prints).

Why no worker movement messages?

Worker movement log lines are intentionally suppressed now. Use diagnostics lines to confirm tuning context.

For implementation details and control flow diagrams, read ARCHITECTURE.md.